HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

Lab 11: Coupling and Cohesion

ITSS Software Development: IT4549E

Students	Student ID
Bùi Trung Hiếu	20235932
Nguyễn Hữu Tuấn Tú	20236005
Nguyễn Tuấn Khanh	20235952
Nguyễn Trần Bình	20235899
Trần Trung Nghĩa	20235983

Semester 20252

1. Coupling
    1.1. Content Coupling

Related modules	Description	Improvement Direction
Client components and product.service	Client interact directly to service instead of through method, violate encapsulation	Keep attribute private, write function for access allowed Data.
View Product Detail module	No content coupling detected. Classes do not access or modify another class's internal state directly; communication is through explicit method calls and return values.	No change needed. Keep product data encapsulated and avoid returning mutable internal objects directly.
VietqrController, VietqrService, PaymentService	No content coupling detected. Controller does not access internal state of services directly; it communicates through public methods such as generateQrCode() and confirmTransactionFromVietqrCallback().	No change needed. Keep controller as API boundary and avoid accessing service private attributes directly.
VietqrService and VietqrHttpClient	No content coupling detected. VietqrService depends on VietqrClient interface, not concrete HTTP implementation internals.	No change needed. Keep using the client interface/port to hide HTTP implementation details.
VietqrCallbackAuthGuard and VietqrInboundService	No content coupling detected. Guard validates token through public method validateAccessToken() instead of reading	No change needed. Keep authentication validation encapsulated inside VietqrInboundService.

	token-generation internals.	

1.2. Common Coupling

Related modules	Description	Improvement Direction
ProductDetailService with ProductDetailRepository and ProductMediaRepository	No common coupling detected. The module does not use global/shared mutable state. Repository dependencies are provided through constructor interfaces.	No change needed. When integrating with NestJS providers, keep dependency injection instead of static/global repositories.
VietqrService, VietqrHttpClient, PaymentService, VietqrCallbackAuthGuard with process.env	Mild common coupling. Several classes read shared environment variables directly, such as VietQR credentials, base URL, callback token, return URL, and transaction type. Although this is configuration data, it is still global shared state.	Move all VietQR configuration access into a dedicated ConfigService/VietqrConfigService. Inject config values instead of reading process.env in many classes.
VietqrInboundService with ConfigService	No problematic common coupling. Environment values are accessed through ConfigService instead of scattered direct access.	Keep this style and apply it to outbound VietQR service as well.
CartComponent, PlaceOrderFEService and CartService	Multiple Angular components (CartComponent and PlaceOrderFEService) all access and modify the same shared cart state through CartService. If CartService were referenced as a public	Apply Angular Singleton pattern: @Injectable(providedIn: 'root'). Inject CartService via constructor DI in every component that needs it. Never use a global or static CartService reference.

	static object, this would be classic common coupling.	
ShippingConstants used by ShippingFeeUtil (FE) and ShippingFeeCalculator (BE)	Both frontend ShippingFeeUtil and backend ShippingFeeCalculator reference the same set of shipping constant values (VAT_RATE, FREE_SHIP_THRESHOLD, base fees). They share a global data definition.	Acceptable common coupling since ShippingConstants are read-only and never mutated at runtime. Ensure all values are declared as static readonly constants. No structural redesign required.

1.3. Control Coupling

Related modules	Description	Improvement Direction
Product-services and Product entities	Need pass a flag to decide behavior of method	Delete dependency in flag. Use polymorphism to build handler class for each product type.
ProductDetailMapper.mapProductDetail()	Mild control coupling. The mapper reads the product type discriminator and uses an if-chain to choose BOOK, NEWSPAPER, CD, or DVD mapping behavior.	Replace the if-chain with Strategy/Factory or a type-to-mapper registry if product types continue to grow.
PaymentService and VietQR/PayPal payment methods	Control coupling detected. requestPayment() checks paymentMethod === PaymentMethod.VIETQR to decide whether to call VietqrService or PaypalService.	Replace payment-method if-chain with Strategy/Factory pattern, such as VietqrPaymentHandler and PaypalPaymentHandler, if more payment methods are added.

PlaceOrderController and PlaceOrderBEService at payment method decision	The controller may pass a paymentMethod string flag ('VIETQR' or 'PAYPAL') into the service, causing the service to branch its internal behavior based on an externally provided flag. This is control coupling.	Separate payment flows into dedicated controllers and services per payment method (PayVietQRController, PayPayPalController). Apply Strategy Pattern so the correct handler is selected without if-else branching inside the service.
DeliveryInfoValidator — internal branching per field validation rule	Inside DeliveryInfoValidator, each field validation uses conditional branches to decide which rule applies (if phone format invalid, if name contains digits, etc.). The caller does not pass flags, but control flow determines validation behavior.	Acceptable at current scope. To improve, extract each rule into a separate ValidationRule class and compose them in a pipeline pattern, removing all internal if-else branching from the main validate() method.
PaymentService and VietQR callback transType	Mild control coupling. confirmTransactionFromVietqrCallback() checks callback.transType to decide whether the callback is acceptable.	Acceptable because this is provider validation. If callback rules grow, move validation into a dedicated VietqrCallbackValidator.
VietqrService.generateQrCode() and auth error retry	Mild control coupling. The service checks whether the error is authentication-related, then refreshes token and retries QR generation.	Acceptable. If retry behavior becomes more complex, extract it into a token-aware VietQR client wrapper.

1.4. Stamp Coupling

Related modules	Description	Improvement Direction
Product-service and dtos	Business logic function input is a complex object meanwhile just use 1 or 2 fields	Just pass necessary argument for function execution instead of passing the whole object.
ProductDetailMapper and product object	Stamp coupling. Type-specific mapper methods receive the full product object but use only fields needed for that product type. This is intentional because the actual product shape may be nested through printableProduct or discProduct.	Acceptable by design. For stricter typing, introduce BookProduct/CDProduct/DVDProduct/NewspaperProduct interfaces and pass typed product data.
ProductDetailAccessChecker with ProductDetailActor and product object	Stamp coupling. canViewProductDetail() receives actor and product objects but currently uses only role, isAuthenticated, and product status.	Acceptable for role-rule extensibility. If no new rules are expected, pass only role, isAuthenticated, and status.
ProductAvailabilityService with product and ProductDetailActor	Stamp coupling. getAvailabilityStatus() receives product and actor objects but uses only status, stock/quantity, and role.	If the rule remains simple, pass only status, stock/quantity, and role to reduce it to data coupling.
PlaceOrderBEService passing DeliveryInfoDto to ShippingFeeCalculator	PlaceOrderBEService passes the entire DeliveryInfoDto to ShippingFeeCalculator.calculate(), but the calculator only needs	Change method signature to: calculate(weight: number, province: string, subtotal: number). Pass only the three required primitive values. This eliminates stamp coupling and

	three fields: weight, province, and subtotal. All other DTO fields are unused stamp data.	makes the method independently testable.
CartComponent passing full CartItem[] to PlaceOrderFEService	CartComponent passes the full CartItem[] array (containing productId, title, quantity, price, weight) to PlaceOrderFEService. However PlaceOrderFEService only needs productId, quantity, price, and weight to call the backend.	Acceptable designer choice since most CartItem fields are actually needed. However, consider mapping CartItem[] to a minimal PlaceOrderItemDto[] before passing to PlaceOrderFEService to be explicit about what the service requires.
VietqrService.generateQrCode( ) and VietqrRequestDto	Stamp coupling. The method receives the full VietqrRequestDto but mainly uses orderId, amount, description, and returnUrl; cancelUrl is currently not used.	Remove unused fields from VietqrRequestDto or use a narrower internal request object when passing data into VietqrService
PaymentService.confirmTransa ctionFromVietqrCallback() and TransactionSyncDto	Stamp coupling. The service receives the whole callback DTO but only uses selected fields such as transType, transactionid, amount, content, transactiontime, referencenumber, and orderId.	Acceptable because the DTO represents an external provider callback. For stricter design, add a mapper from TransactionSyncDto to a smaller internal VietqrCallbackData object.

VietqrHttpClient.generateQRCode() and VietqrGenerateQRCodeRequest	Stamp coupling. The client receives a structured request object for VietQR QR generation.	Acceptable because this object matches the external VietQR API payload. Keep it as a boundary DTO.

1.5. Data Coupling

Related modules	Description	Improvement Direction
ProductDetailValidator.validateProductId()	Data coupling. The method receives only a primitive productId value and has no external dependency or side effect.	Already at the best coupling level.
ProductDetailService to repository methods	Data coupling in repository calls. getProductDetail() passes only the productId string to findProductById() and findMediaByProductId().	Already good. Keep the service independent from concrete database implementation.
CartQuantityValidator.validateAddToCartQuantity()	Data coupling. The method receives only quantity, stock, and productStatus values and returns a boolean validation result.	Already at the best coupling level.
PlaceOrderController and PlaceOrderBEService via typed request/response DTOs	Controller passes only the specific DTO needed per endpoint (DeliveryInfoDto, PlaceOrderRequestDto) and receives back only the required response DTO (InvoicePreviewDto,	No improvement needed. This is data coupling — the ideal coupling level. Always design DTOs with only the fields required for each specific operation to maintain this quality.

	OrderSuccessDto). No excess data crosses the boundary.	
PlaceOrderBEService and all Repositories (OrderRepository, ProductRepository, InvoiceRepository, TransactionRepository)	Service passes only the entity object or primitive ID needed for each repository operation and receives back only the resulting entity. No flags or composite objects are passed across the service-repository boundary.	No improvement needed. Data coupling between service and repositories is the target design. Continue passing only the entity or primitive IDs required per operation.
PlaceOrderFEService and PlaceOrderController via HTTP REST API	PlaceOrderFEService sends only the necessary JSON body per endpoint (CartItemDto[], DeliveryInfoDto) and receives a typed response object (InvoicePreviewDto, OrderSuccessDto). No internal backend objects are exposed across the HTTP boundary.	No improvement needed. The HTTP boundary is the strongest enforcer of data coupling. Maintain by using typed DTOs on both sides and never exposing raw entity structures in API responses.
VietqrClient.getAccessToken()	Data coupling. The method receives only primitive username and password values.	Already good.
VietqrInboundService.validateInboundCredentials()	Data coupling. The method receives only username and password primitives.	Already good.

Vietqr helper functions	Data coupling. Functions such as normalizeVietqrContent() and buildGatewayOrderId() receive primitive string values and return primitive strings.	Already at the best coupling level.
PaymentService to PlaceOrderPaymentPort	Data coupling. PaymentService passes only order/payment lookup fields instead of the whole order object.	Already good. Keep PlaceOrder details hidden behind the port.
PaypalController → PaypalService	Communication is clean data coupling	Already at the best coupling level.

2. Cohesion

2.1. Coincidental Cohesion

Related modules	Description	Improvement Direction
View Product Detail module	No coincidental cohesion detected. Each class has a clear responsibility related to viewing product detail, validation, mapping, access checking, availability, or cart quantity validation.	No change needed. Avoid adding unrelated product-management or order logic to this module.
VietQR module	No coincidental cohesion detected. Classes are related to VietQR QR generation, callback handling, token validation, and provider communication.	No change needed. Avoid adding unrelated order-management or product logic into VietQR module.

2.2. Logical Cohesion

Related modules	Description	Improvement Direction
product-service	All generate product method put into 1 general class but it interact with other repo	Separate into small class service, each service process 1 task (BookHandler, DiscHandler) a
View Product Detail module	No dominant logical cohesion detected. ProductDetailMapper contains several mapping methods, but they all serve one product-detail transformation goal rather than unrelated operations selected by an external flag.	Keep each class focused on one responsibility. If mapper logic grows, split it into per-product-type mapper classes.
PlaceOrderBEService — groups multiple logically related but distinct operations	The service contains checkStock(), processDeliveryInfo(), buildInvoicePreview(), and confirmOrder(). All belong to the Place Order flow but each serves a different logical purpose: stock validation, delivery validation, fee calculation, and order persistence.	Delegate each distinct concern to a specialist class: ShippingFeeCalculator for fee logic, DeliveryInfoValidator for validation. PlaceOrderBEService becomes a pure orchestrator, improving cohesion from logical toward communicational.
VietqrService	Mild logical cohesion. The service contains several VietQR-related operations: access token retrieval, QR generation, callback status mapping, callback response building, and sandbox	Acceptable now. If the class grows, split into VietqrTokenService, VietqrQrService, and VietqrCallbackMapper

	callback testing. They are related, but selected through different public methods.	
PaymentService	Logical cohesion exists because one service handles multiple payment lifecycle operations and branches by payment method.	If payment logic grows, extract provider-specific handlers such as VietqrPaymentHandler and PaypalPaymentHandler

2.3. Temporal Cohesion

Related modules	Description	Improvement Direction
View Product Detail module	No temporal cohesion detected. The module does not group operations merely because they happen at the same time, such as startup, initialization, or cleanup.	No change needed.
PlaceOrderModule NestJS bootstrap — simultaneous initialization of unrelated components	When the NestJS app starts, PlaceOrderModule simultaneously registers the controller, injects all repository dependencies, wires the service, and sets up exception filters. These happen together only because of application startup timing, not logical dependency.	Acceptable temporal cohesion in NestJS module setup. No redesign needed. Framework-driven initialization always exhibits temporal cohesion and it is a known accepted trade-off.
VietQR module	No strong temporal cohesion detected. The module does not group methods only because	No change needed.

	they happen during startup/shutdown.	
VietqrService token cache	Small temporal relation exists because token retrieval, token expiration check, and token clearing must happen in a correct time sequence.	Acceptable because all token lifecycle logic is encapsulated inside VietqrService

2.4. Procedural Cohesion

Related modules	Description	Improvement Direction
ProductDetailService.getProductDetail()	A small procedural sequence exists: validate productId → fetch product → fetch media → map details → compute availability. The class stays cohesive because each step contributes directly to one use-case result.	Keep orchestration thin. If more steps are added, move them into dedicated services instead of expanding this method.
DeliveryInfoValidator.validate() — sequential field validation steps	The validate() method executes steps in a fixed order: validateReceiverName(), then validatePhoneNumber(), then validateProvince(), then validateStreetAddress(). Each step runs sequentially to collect all field errors at once.	Acceptable procedural cohesion for a validator class. To improve, extract each field rule into a ValidationRule interface and compose via a pipeline. This optimization is optional at current project scope.
PlaceOrderBEService.confirmOrder() — sequential persistence steps	confirmOrder() runs in a fixed order: validate input, save OrderEntity (produces order.id),	Extract each step into a clearly named private method (saveOrder, saveInvoice, saveTransaction, sendEmail).

	save InvoiceEntity using order.id (produces invoice.id), save TransactionEntity using invoice.id, then send confirmation email.	Keep them in the same service class since the sequential dependency between steps must be preserved.
PaymentService.requestPayment() for VietQR	Procedural cohesion. The method follows a sequence: get payment context → validate amount → create transaction → build gateway order ID → generate QR → update transaction → return result.	Keep orchestration thin. If more VietQR-specific steps are added, move them to a VietQR payment handler
VietqrController.syncTransaction()	Procedural cohesion. The method receives callback → confirms transaction → builds success response, or catches error → builds error response.	Acceptable because this is controller-level request handling. Keep business validation inside PaymentService/VietqrService.
VietqrInboundService.generateAccessToken()	Procedural cohesion. It parses Basic Auth → validates credentials → builds payload → signs token → returns token response.	Acceptable because all steps contribute to one token-generation use case.

2.5. Communicational Cohesion

Related modules	Description	Improvement Direction
ProductDetailMapper	Communicational cohesion. Mapping methods operate on the same kind of input data (product data) and return product-detail response data.	Acceptable for this use case. If type-specific mapping becomes complex, split into BookDetailMapper, CDDetailMapper, DVDDetailMapper, and NewspaperDetailMapper.

ProductDetailAccessChecker	Currently functional because it has one access-check method. It may become communicational if future view/edit/delete checks are added around the same actor and product inputs.	Keep this class limited to View Product Detail access rules; create separate checkers for other use cases.
PlaceOrderBEService — buildInvoicePreview() and confirmOrder() operating on the same invoice data	Both buildInvoicePreview() and confirmOrder() work on the same InvoicePreviewDto data structure. buildInvoicePreview() produces it and confirmOrder() consumes and persists it. They share the same data domain.	Communicational cohesion is acceptable here since all methods operate on the same invoice data. Keep InvoicePreviewDto focused. If invoice data grows, consider splitting into a preview DTO and a persistence DTO.
OrderRepository, InvoiceRepository, ProductRepository, TransactionRepository — all methods operate on the same entity	All methods within each repository class (save, findById, updateStatus, findByOrderId) operate on the same entity and the same database table, sharing the same data source — communicational cohesion.	Communicational cohesion is the expected and ideal cohesion level for a repository class. All repositories in the Place Order design correctly exhibit this level. No improvement needed.
VietqrService	Communicational cohesion. Methods operate around the same VietQR provider data: access token, QR request, QR response, and callback response.	Acceptable. Split only if provider logic becomes large.

VietqrHttpClient	Communicational cohesion. All methods communicate with the VietQR HTTP API using the same base URL and authentication style.	Already good.
Vietqr DTOs	Communicational cohesion. Each DTO groups data for one VietQR communication boundary, such as QR request or transaction callback.	Already good.

2.6. Sequential Cohesion

Related modules	Description	Improvement Direction
ProductDetailMapper	Sequential cohesion. mapGeneralProductDetail() creates the base output, then type-specific mapping methods reuse that output and add Book, Newspaper, CD, or DVD fields.	Acceptable because the chain is short and predictable. Add explicit return types or per-type mappers if the mapper grows.
PlaceOrderBEService.confirmOrder() — output of each step is input to the next	confirmOrder() runs: save(OrderEntity) produces order.id used to build InvoiceEntity; save(InvoiceEntity) produces invoice.id used to build TransactionEntity; all three are then used to send the confirmation email. True sequential dependency.	Sequential cohesion is appropriate for this method since steps are genuinely dependent. Extract each step into a clearly named private method for readability. Do not split into separate classes as that would obscure the sequential dependency.

VietqrService.generateQrCode( )	Sequential cohesion. The access token is obtained first, then QR request is built, then VietQR API is called, then response is mapped to QRCodeDataDto.	Acceptable because the chain is short and directly supports QR generation
PaymentService.confirmTransa ctionFromVietqrCallback()	Sequential cohesion. It validates callback type → checks duplicate → finds transaction → validates amount/status → updates transaction → notifies PlaceOrder.	Acceptable. If callback processing grows, extract duplicate detection and transaction matching into helper services.
PlaceOrderFEService — checkStock(), submitDeliveryInfo(), confirmOrder() forming a sequential checkout pipeline	The three service methods form a sequential pipeline: checkStock() must succeed before submitDeliveryInfo() is called; submitDeliveryInfo() must return a valid InvoicePreview before confirmOrder() can execute.	Sequential cohesion is natural and correct for a multi-step checkout flow. Enforce the sequence using Angular component state or route guards to prevent users from skipping steps. No structural redesign needed.
The token → authenticated call pipeline in PaypalService	shows clean sequential cohesion	Acceptable

2.7. Functional Cohesion

Related modules	Description	Improvement Direction
ProductDetailValidator	Functional cohesion. The class has one purpose: validate the product identifier format.	Already optimal.

ProductAvailabilityService	Functional cohesion. The class computes the availability state and whether the actor can add the product to cart.	Already good. Keep pricing, access, and media logic outside this class.
CartQuantityValidator	Functional cohesion. The class has one purpose: validate whether the requested add-to-cart quantity is valid for the current stock and product status.	Already optimal.
ProductDetailService	Functional cohesion. The service orchestrates exactly one use-case result: a complete product-detail response for a given product and actor.	Already good. If the use case grows, extract new collaborators while keeping this service as a thin coordinator.
VietqrHttpClient	Functional cohesion. The class has one purpose: communicate with VietQR outbound HTTP APIs.	Already good.
VietqrInboundService	Functional cohesion. The class has one purpose: handle inbound VietQR authentication and callback token validation.	Already good.
VietqrCallbackAuthGuard	Functional cohesion. The class has one responsibility: protect VietQR callback	Already good.

	endpoint using bearer token validation.	
Vietqr normalize helpers	Functional cohesion. Helper functions only normalize VietQR-safe content and order identifiers.	Already optimal.
ConfigService	Functional cohesion. The class only reads configuration values by key.	Already good, but should be reused more broadly in VietQR services.
ShippingFeeCalculator — all methods exclusively serve the single purpose of computing shipping fee	ShippingFeeCalculator contains only calculate() and applyFreeShipDiscount (). Every line of code contributes exclusively to computing one output: the delivery fee number. No unrelated logic exists anywhere in the class.	No improvement needed. ShippingFeeCalculator exhibits functional cohesion — the highest cohesion level. Maintain this by strictly refusing to add any logic unrelated to shipping fee calculation to this class.
DeliveryInfoValidator — all methods exclusively serve the single purpose of validating delivery information	Every method in DeliveryInfoValidator (validate, validateReceiverName, validatePhoneNumber, validateProvince, validateStreetAddress) contributes exclusively to validating delivery fields and producing a single ValidationResult.	No improvement needed. The class has functional cohesion. Keep all delivery validation rules within this class only. Never add shipping fee logic, order creation logic, or any unrelated concern here.

Each Repository class (OrderRepository, ProductRepository, InvoiceRepository, TransactionRepository)	Each repository class contains only methods that perform data access on its corresponding single database table. OrderRepository only touches the orders table, ProductRepository only the product table. No cross-table queries.	No improvement needed. All repository classes correctly exhibit functional cohesion. Enforce the single-table-per-repository rule and never allow a repository to query across multiple unrelated tables.
PaypalModule	Has excellent functional cohesion (minimal, focused wiring)	No improvement needed. All repository classes correctly exhibit functional cohesion.
PaypalController	a pure routing layer with no business logic — textbook functional cohesion	No improvement needed. All repository classes correctly exhibit functional cohesion.